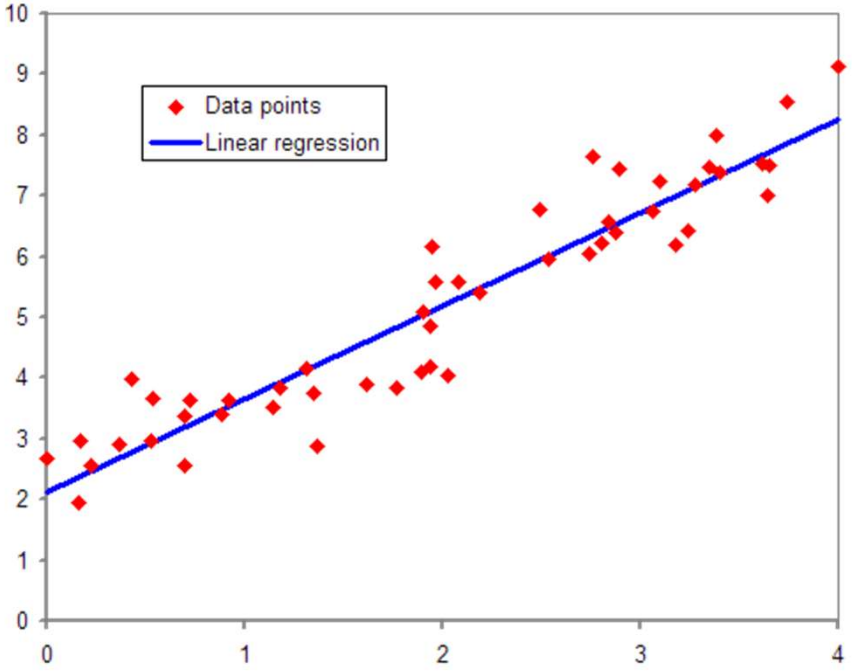
REGRESSION

REGRESSION

❖ What is regression?

- ✓ Predicting the target y given a D-dimensional vector x.
- ✓ or estimating the relationship between x and y.
- ✓ or finding a function from x to y.





니, 그래쒀?

REGRESSION

❖ What is regression?

- ✓ Predicting the target y given a D-dimensional vector x.
- ✓ or estimating the relationship between x and y.
- ✓ or finding a function from $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)$ to y.

❖ Linear regression

- ✓ Linear regression attempts to find the relationship (function) between two variables by fitting a linear equation to observed data.

$$y \cong \hat{y} = w_0 + w_1 x_1 + \ldots + w_p x_p = f(\boldsymbol{x}) \text{ or } f_{\boldsymbol{w}}(\boldsymbol{x})$$

$$\boldsymbol{w} = (w_1, \ldots, w_p) \text{ as coefficients or weights and } w_0 \text{ as intercept}$$

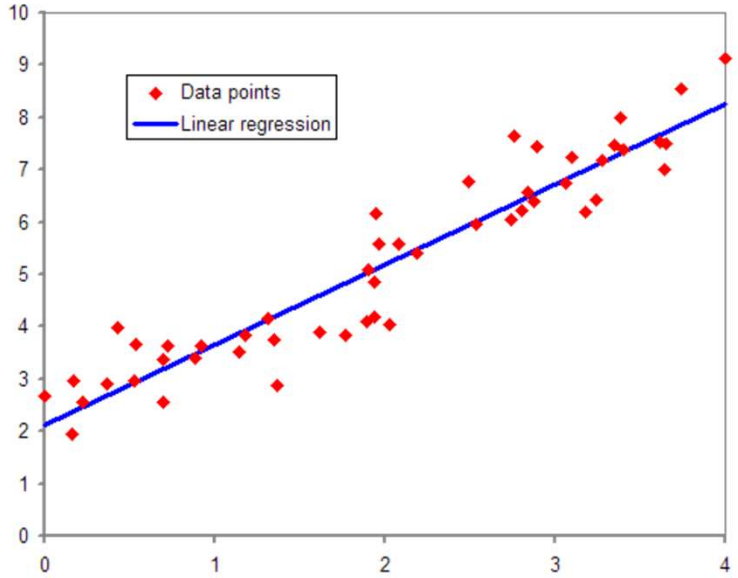
- ✓ $\hat{y}$ is called the estimated y or the predicted y according to $f_{\boldsymbol{w}}(\boldsymbol{x})$
- ✓ Our goal is to make $\hat{y}$ as similar as possible to y for all x.

❖ Loss function E

- ✓ It indicates the performance of the estimation.
- ✓ Given observed input and output pairs (ground truth),

$$Data = \{(x(1), y(1)), (x(2), y(2)), \dots, (x(N), y(N))\}$$
$$\text{or } Data\text{: } X = \{x(1), x(2), \dots, x(N)\} \text{ and } Y = \{y(1), y(2), \dots, y(N)\}$$

the loss function can be simply designed to minimize errors defined by



$$E(\boldsymbol{w}) = \sum_{n=1}^{N} \left(y(n) - \hat{y}(n)\right)^2 = \sum_{n=1}^{N} \left(y(n) - f_{\boldsymbol{w}}(\boldsymbol{x}(n))\right)^2$$

$$y \cong \hat{y} = w_0 + w_1 x_1 + \dots + w_p x_p = f(\boldsymbol{x}) \text{ or } f_{\boldsymbol{w}}(\boldsymbol{x})$$

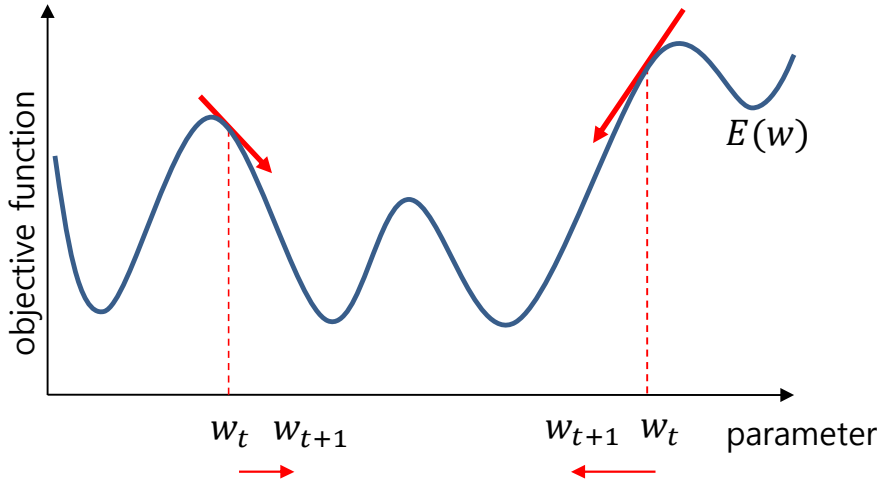
❖ Learning

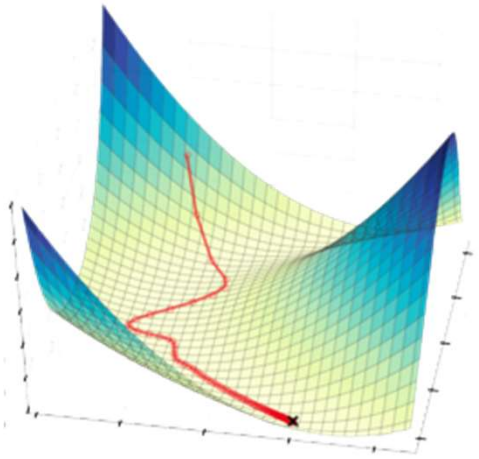
✓ Learning is to update the parameter by

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \nabla_{\boldsymbol{w}} E(\boldsymbol{w})$$

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} \left(y(n) - \hat{y}(n)\right)^2 = \sum_{n=1}^{N} \left(y(n) - f_{\boldsymbol{w}}(\boldsymbol{x}(n))\right)^2$$

- gradient $\nabla_{\boldsymbol{w}} E(\boldsymbol{w})$ shows the direction for the parameter update.
- learning rate ϵ determines how far the weight should move.





❖ A simple problem

- ✓ Given X and Y, what is the exact number y for $x = 4$?
  - $X = [1, 2, 3]$
  - $Y = [2, 4, 6]$
- ✓ Obvious answer is a number $y = 8$.

- ✓ What is the model for the problem?
- ✓ Given a simple model $\hat{y} = w_0 + w_1 x_1$
- ✓ Based on the data X and Y, the possible model (with parameters) would be $y = 2x$

- ✓ But how does a **computer** find the model?

❖ A simple problem

✓ Let's try to find $\boldsymbol{w}$ for a simple linear model $f_{\boldsymbol{w}}(\boldsymbol{x})$

```
import matplotlib.pyplot as plt

x_data = [1.0, 2.0, 3.0]
y_data = [2.0, 4.0, 6.0]

def predict(w, x):
  return x*w;

def errorSquared(y, y_pred):
  return (y - y_pred)*(y - y_pred)

def loss(input):
  return sum(input)

# learinng
w = 0.00 # initial weight
for epoch in range(20):
  error = []
  for x, y in zip(x_data, y_data):
    y_pred = predict(w, x)
    error.append(errorSquared(y, y_pred))
  l = loss(error)
  w = w + 0.3
  print("Epoch", epoch, "\tw= ", '%.2f'%w, "\tloss= ", '%.2f'%l)
  plt.scatter(w, l, color='r')
```

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \nabla_{\boldsymbol{w}} E(\boldsymbol{w})$$

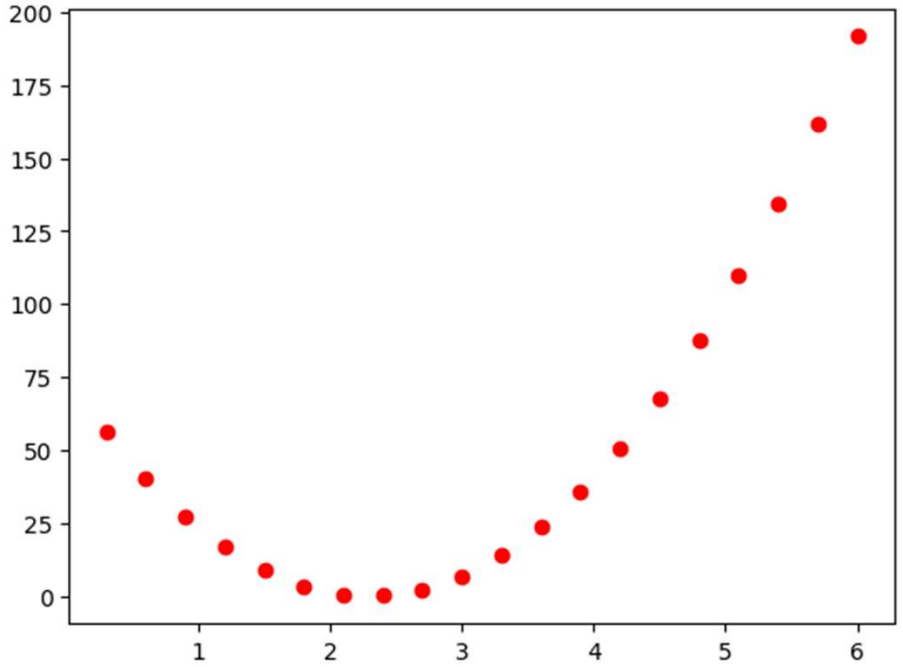
$$\downarrow$$

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + 0.3$$

❖ A simple problem

✓ Let's try to find $\boldsymbol{w}$ for a simple linear model $f_{\boldsymbol{w}}(\boldsymbol{x})$

Epoch 0	w= 0.30	loss= 56.00
Epoch 1	w= 0.60	loss= 40.46
Epoch 2	w= 0.90	loss= 27.44
Epoch 3	w= 1.20	loss= 16.94
Epoch 4	w= 1.50	loss= 8.96
Epoch 5	w= 1.80	loss= 3.50
Epoch 6	w= 2.10	loss= 0.56
Epoch 7	w= 2.40	loss= 0.14
Epoch 8	w= 2.70	loss= 2.24
Epoch 9	w= 3.00	loss= 6.86
Epoch 10	w= 3.30	loss= 14.00
Epoch 11	w= 3.60	loss= 23.66
Epoch 12	w= 3.90	loss= 35.84
Epoch 13	w= 4.20	loss= 50.54
Epoch 14	w= 4.50	loss= 67.76
Epoch 15	w= 4.80	loss= 87.50
Epoch 16	w= 5.10	loss= 109.76
Epoch 17	w= 5.40	loss= 134.54
Epoch 18	w= 5.70	loss= 161.84
Epoch 19	w= 6.00	loss= 191.66



At 7th epoch, $w = 2.40$ yields the lowest loss.
But is it optimal?

❖ A simple problem

✓ Gradient decent approach

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \nabla_{\boldsymbol{w}} E(\boldsymbol{w})$$

$$\uparrow$$

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + 0.3$$

✓ The loss function we simply defined is

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} (y(n) - \hat{y})^2 = \sum_{n=1}^{N} \Big(y(n) - f_{\boldsymbol{w}}\big(x(n)\big)\Big)^2$$

✓ The gradient of the loss function is

$$\nabla_{\boldsymbol{w}} E(\boldsymbol{w}) =$$

$$= \sum_{n=1}^{N} -2 \cdot (y(n) - x_1(n) \cdot w_1) \cdot x_1(n)$$

❖ A simple problem

✓ Let's try to find $\boldsymbol{w}$ for a simple linear model $f_{\boldsymbol{w}}(\boldsymbol{x})$

```
def gradient(x, y):
  return -2*x*(y - x*w)

# learinng
w = 0.00 # initial weight
for epoch in range(20):
  error = []
  grad = []
  for x, y in zip(x_data, y_data):
    y_pred = predict(w, x)
    error.append(errorSquared(y, y_pred))
    grad.append(gradient(x,y))
  l = loss(error)
  w = w -0.005*sum(grad)
  print("Epoch", epoch, "\tw= ", '%.2f'%w, "\tloss= ", '%.2f'%l)
  plt.scatter(w, l, color='r')
```

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \nabla_{\boldsymbol{w}} E(\boldsymbol{w})$$

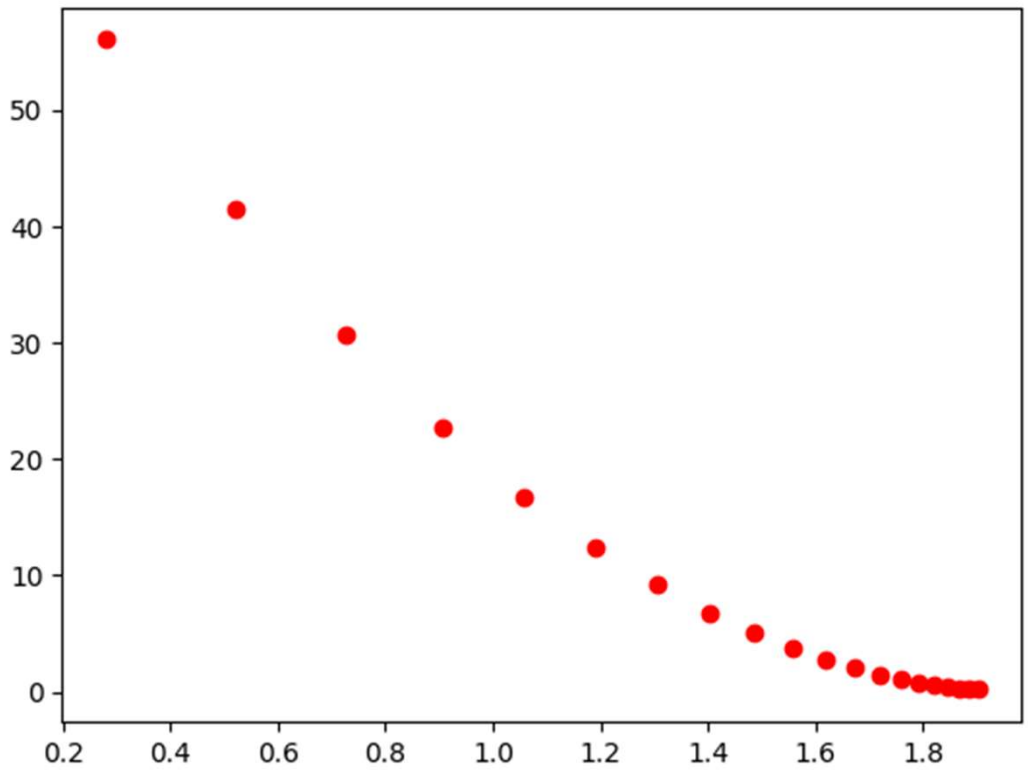
$$\nabla_{\boldsymbol{w}} E(\boldsymbol{w}) = \sum_{n=1}^{N} -2 \cdot (y(n) - x_1(n) \cdot w_1) \cdot x_1(n)$$

$$\epsilon = 0.005$$

❖ A simple problem

✓ Let's try to find $\boldsymbol{w}$ for a simple linear model $f(\boldsymbol{w}, \boldsymbol{x})$

```
Epoch 0     w=  0.28     loss=  56.00
Epoch 1     w=  0.52     loss=  41.42
Epoch 2     w=  0.73     loss=  30.63
Epoch 3     w=  0.91     loss=  22.66
Epoch 4     w=  1.06     loss=  16.76
Epoch 5     w=  1.19     loss=  12.39
Epoch 6     w=  1.30     loss=  9.17
Epoch 7     w=  1.40     loss=  6.78
Epoch 8     w=  1.49     loss=  5.01
Epoch 9     w=  1.56     loss=  3.71
Epoch 10    w=  1.62     loss=  2.74
Epoch 11    w=  1.67     loss=  2.03
Epoch 12    w=  1.72     loss=  1.50
Epoch 13    w=  1.76     loss=  1.11
Epoch 14    w=  1.79     loss=  0.82
Epoch 15    w=  1.82     loss=  0.61
Epoch 16    w=  1.85     loss=  0.45
Epoch 17    w=  1.87     loss=  0.33
Epoch 18    w=  1.89     loss=  0.25
Epoch 19    w=  1.90     loss=  0.18
```



❖ A simple problem

✓ Test the model

– After 20 epochs

```
print("Predicted value (after training)", "When x=4, y will be : ", predict(w, 4))
print("Predicted value (after training)", "When x=6, y will be : ", predict(w, 6))
```

```
Predicted value (after training) When x=4, y will be :  7.608204889453318
Predicted value (after training) When x=6, y will be :  11.412307334179976
```

– After 100 epochs

```
Predicted value (after training) When x=4, y will be :  7.999997746097573
Predicted value (after training) When x=6, y will be :  11.999996619146359
```

✓ More training may produce the exact answer.

❖ Including Intercept

✓ The function we assumed is

$$f_{\boldsymbol{w}}(x) = w_1 \cdot x_1$$

✓ Let's include intersection and now we have two parameters w_0, w_1 to find

$$f_{\boldsymbol{w}}(x) = w_0 + w_1 \cdot x_1$$

✓ Then, the gradient of the loss function becomes two separate equations:

$$\nabla_{\boldsymbol{w}} E(\boldsymbol{w}) = \nabla_{\boldsymbol{w}} \left(\sum_{n=1}^{N} \left(y(n) - f_{\boldsymbol{w}}(x(n)) \right)^2 \right) = \nabla_{\boldsymbol{w}} \left(\sum_{n=1}^{N} \left(y(n) - \left(w_0 + w_1 \cdot x_1(n) \right) \right)^2 \right)$$

$$\nabla_{w_0} E(\boldsymbol{w}) = \nabla_{w_0} \left(\sum_{n=1}^{N} \left(y(n) - \left(w_0 + w_1 \cdot x_1(n) \right) \right)^2 \right) =$$

$$\nabla_{w_1} E(\boldsymbol{w}) = \nabla_{w_1} \left(\sum_{n=1}^{N} \left(y(n) - \left(w_0 + w_1 \cdot x_1(n) \right) \right)^2 \right) =$$

❖ Including Intercept

✓ Weight update

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \nabla_{\boldsymbol{w}} E(\boldsymbol{w}) \rightarrow (w_0)_{t+1} = (w_0)_t - \epsilon \nabla_{w_0} E(\boldsymbol{w}),\ (w_1)_{t+1} = (w_1)_t - \epsilon \nabla_{w_1} E(\boldsymbol{w})$$

```
def predict2(w, x):
  return w[0] + w[1] * x;

# learinng
w = [0.00, 0.00] # initial weight [w_0, w_1]
epochs = 200
learning_rate = 0.01
for epoch in range(epochs):
  error = []
  grad_w0 = []
  grad_w1 = []
  for x, y in zip(x_data, y_data):
    y_pred = predict2(w, x)
    error.append(errorSquared(y, y_pred))
    grad_w0.append(-2*(y - (w[0] + x*w[1])))
    grad_w1.append(-2*(y - (w[0] + x*w[1]))*x)
  l = loss(error)
  w[0] = w[0] -learning_rate*sum(grad_w0)
  w[1] = w[1] -learning_rate*sum(grad_w1)

  print("Epoch", epoch, "\tw_0= ", '%.2f'%w[0], "\tw_1= ", '%.2f'%w[1], "\tloss= ", '%.2f'%l)
```

❖ Exercise 1 – speed of sound

- ✓ Speed of sound at temperature
- ✓ Based on the data (temperature, speed),
  what is the prediction of speed at temperature of 20°C.

Temperature X(℃)	Speed Y(m/s)
0	331
1	331.6
2	332.2
3	332.8

```
x_data = [0.0, 1.0, 2.0, 3.0]
y_data = [331.0, 331.6, 332.2, 332.8]
```